Prueba técnica

Tiempo estimado: 3-4 días
Lenguaje sugerido: Java
Framework sugerido: Spring Boot
Entrega: Repositorio en GitHub con instrucciones para ejecutar el proyecto

VISIÓN GENERAL

Desarrollar un sistema distribuido que permite crear pedidos y reservar productos sin vender más unidades de las disponibles.

La solución debe soportar múltiples solicitudes concurrentes, instancias, mensajes duplicados, errores temporales y comunicación asincrónica entre servicios.

OBJETIVOS

La solución debe contar con al menos dos microservicios desarrollados con Spring Boot.

- Order Service: Responsable de:
    - Crear pedidos.
    - Consultar el estado de un pedido.
    - Cancelar pedidos.
    - Publicar eventos relacionados con pedidos.
    - Actualizar el estado del pedido según las respuestas recibidas.
    - Estados posibles:
        - PENDING
        - CONFIRMED
        - REJECTED
        - CANCELLED
    - Operaciones idempotentes
- Inventory Service: Responsable de:
    - Mantener el stock disponible de cada producto.
    - Reservar stock.
    - Liberar stock por cancelaciones.
    - Evitar que el stock quede negativo.
    - Procesar correctamente eventos repetidos.

    - Operaciones idempotentes

Cada microservicio debe tener su propia base de datos o esquema. No se permite compartir tablas entre los servicios.

- Sistema de mensajería
    - El candidato puede utilizar Kafka, RabbitMQ u otra alternativa equivalente.
    - La elección debe estar explicada en el README, incluyendo:
        - Motivo de la elección.
        - Garantía de entrega utilizada.
        - Estrategia de reintentos.
        - Manejo de mensajes que no pueden procesarse.
        - Orden de procesamiento, cuando corresponda.

ESPECIFICACIONES

Endpoints mínimos

Order Service:

- Carga de pedido: **POST /orders**
- Consulta de pedido: **GET /orders/{orderId}**
- Cancelación de pedido: **POST /orders/{orderId}/cancel** o **DELETE /orders/{orderId}**

Inventory Service

- Carga de Stock: **POST /products**
- Consulta de Stock: **GET /products/{productId}/stock**

Idempotencia en endpoints de escritura. Bloqueo distribuido.

Mensajes duplicados

El sistema de mensajería debe considerarse de tipo at-least-once. Un mismo evento puede llegar dos o más veces.

Condiciones de carrera

Un pedido puede ser cancelado mientras la reserva de stock se encuentra en procesamiento. El sistema debe garantizar que el estado final sea consistente.

ENTREGABLES

- Repositorio con:
    - Código fuente
    - Instrucciones para desplegar y probar el proyecto
    - Tests unitarios
    - Stack de pruebas de integración con Postman, Bruno, Insomnia o similares.
    - Pruebas concurrentes
- Bonus:
    - Patrones de diseño
    - Arquitectura
    - Observabilidad y trazabilidad
    - Configuración de agentes (skills, commands, rules, AGENTS/CLAUDE.md, TDD, SDD, etc)

No se considera un requisito implementar todos los puntos bonus.